DUAL: Neighbourhood Attention and Dynamic Kernel Mixing for Image Restoration

Rift73
github.com/Rift73/DUAL
Technical report - 1 November 2026

Abstract

DUAL is a single-image restoration backbone that combines local window attention, Neighbourhood Attention (NA), regional channel mixing, cross-scale self-exemplar retrieval, and image-conditioned convolution. Local layers use anchored windows with tabulated relative position bias. Context layers use NA: each query pixel attends to its own 32 × 32 or 64 × 64 in-bounds neighbourhood, with stationary Fourier bias represented through query/key factor concatenation. A separate regional operator blends channel interactions across four shifted grids. Cross-scale retrieval reuses a shallow encoder on a filtered coarse image and transfers packed values into the trunk and reconstruction stream. At each group exit, a centered router mixes residual kernels into a reparameterized static base before convolution. Six presets support 1x, 2x, and 4x restoration. This report details their architecture, scale-dependent reconstruction, initialization, and training and deployment behavior.

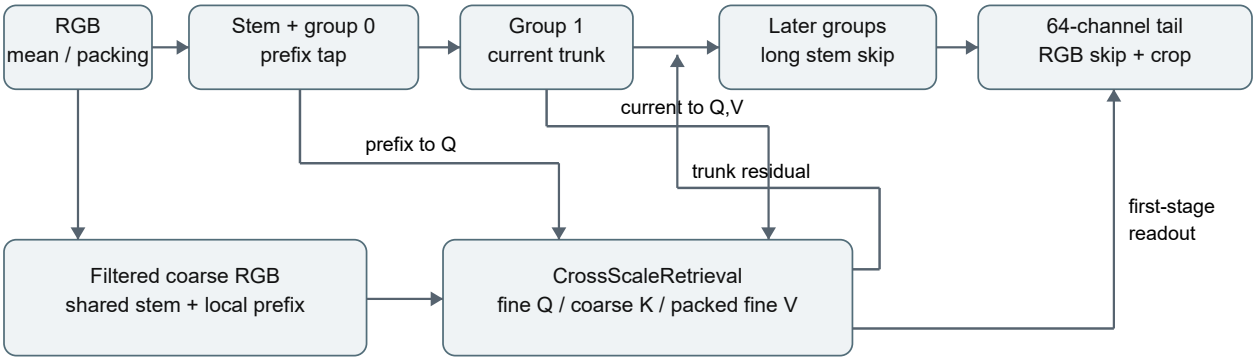


Figure 1. DUAL data flow, with zero-based groups. CrossScaleRetrieval reads after group 1; SecondOctaveTransfer uses new values from the first reconstruction octave (a 2x spatial expansion). Arrows simplify residual and normalization paths.

1 Design rationale

The name DUAL emphasizes complementary spatial and regional channel attention. The architecture separates three types of adaptation: spatial attention chooses relationships between locations, regional attention mixes channels, and a global image summary changes group-exit kernels. These mechanisms have different supports and cannot be treated as interchangeable forms of attention. The design draws on restoration transformers, rank-factorized bias, cross-scale self-similarity, and dynamic convolution [1-6].

2 Model family and residual hierarchy

For RGB input x of shape B x 3 x H x W and requested scale s in {1, 2, 4}, DUAL returns B x 3 x sH x sW. It subtracts the fixed RGB mean (0.4488, 0.4371, 0.4040), extracts stem features, processes residual groups, and adds a long stem skip after final token normalization and trunk convolution. The output combines the learned RGB residual with a nearest-neighbor image skip and restores the mean.

$$y = \mathrm{crop}_{sH,sW}\left[R_s(x - \mu) + NN_s(x - \mu) + \mu \right]. \quad (1)$$

Factory	C	Heads	Group depths	Layers	PU2
dual_light	128	4	4, 3, 3, 6	16	Yes
dual_xs	128	4	4, 3, 3, 6	16	No
dual_s	160	5	6 groups x 6	36	No
dual_m	192	6	6 groups x 6	36	No
dual_l	224	7	10 groups x 6	60	No
dual_xl	256	8	12 groups x 6	72	No

Table 1. Released preset geometry. Head width is 32 and reconstruction width is 64. Layer counts exclude the extra execution of the shared coarse-image prefix and are not parameter counts.

2.1 Local/context schedule

A full group follows [G16, shifted G16, C32, G16, shifted G16, C64], followed by a dynamic 3 × 3 write-back and a group residual. G16 denotes local spatial attention with a 16-pixel window; C32 and C64 use Neighbourhood Attention with support widths 32 and 64, not channel counts. G layers apply spatial attention and a convolutional feed-forward network. C layers insert regional channel attention between the NA and feed-forward residuals. Light and XS retain original slot indices (0,1,2,3), (3,4,5), (0,1,2), and (0,1,2,3,4,5). They therefore retain 11 local and five NA context layers, including both context sizes. NA is enabled by default across all six presets and all supported output scales.

2.2 Light input packing

Light rearranges 2 × 2 mean-shifted RGB neighborhoods into 12 channels before the stem. Its trunk grid is half the input resolution and its internal scale is 2s. The input is padded to a multiple of four, using reflection where legal and replication for very small dimensions, then cropped after reconstruction. XS has the same width and selected layers but does not pack the input. Consequently identical numeric window sizes refer to different input-pixel distances in Light and XS.

The raw DUAL constructor defaults to C96 with four full groups. It is not the Light preset. All six named factories support 1x/2x/4x; reconstruction weights are scale-specific.

3 Local window attention and Neighbourhood Attention

Neighbourhood Attention (NA) is query-centred spatial attention [3]. A local-window layer assigns several queries to a shared window; an NA layer instead selects an in-bounds donor rectangle for each query pixel. Adjacent queries can therefore retrieve values across a local-window boundary without averaging multiple output predictions. DUAL combines this support rule with even-sized 32 × 32 and 64 × 64 neighbourhoods and learned relative Fourier bias; its implementation does not require the NATTEN library.

Each spatial sublayer receives channel-normalized features. Its attention output is multiplied by a sigmoid gate computed with a depthwise 3 × 3 and pointwise convolution, then projected and added residually. Local G16 layers alternate unshifted and half-window-offset ownership. At a boundary, window origins are clamped inward; each output pixel still has one designated owner. Short axes use their available extent.

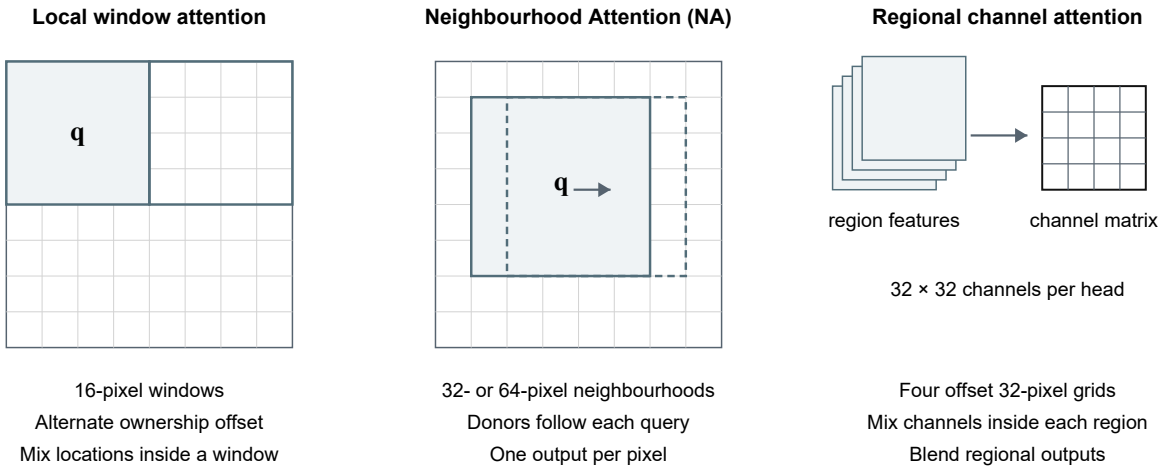


Figure 2. Three different operations. Local window attention and Neighbourhood Attention (NA) mix spatial locations; regional attention instead builds channel-correlation matrices within regions. These are structural schematics, not measured attention maps.

$$A_p = \Sigma_{j \in S(p)} \, \mathrm{softmax}_j(q_p k_j^T / \sqrt{32} + B(p,j)) \, v_j. \quad (2)$$

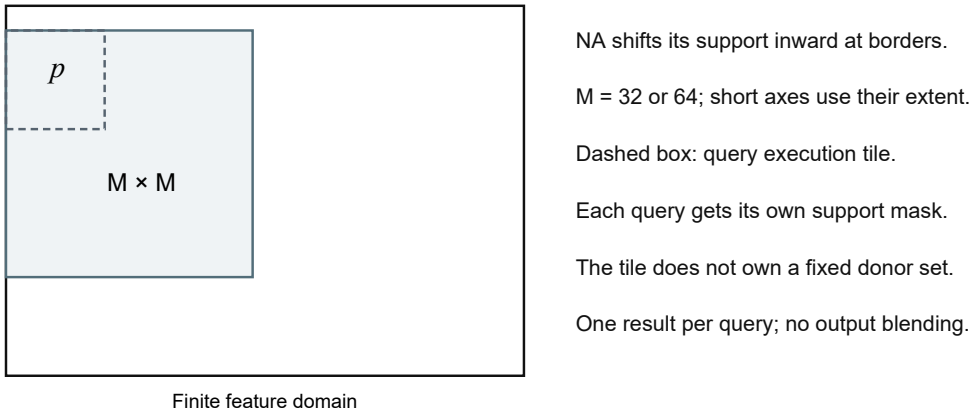


Figure 3. Neighbourhood Attention (NA). Each query selects an in-bounds support that follows its position and shifts inward at borders. Adjacent supports may coincide near an edge. Execution tiles share gathered storage, not output predictions.

For context size M, define kH=min(H,M), kW=min(W,M), aY(p)=clamp(pY-floor(kH/2),0,H-kH), and the analogous aX. S(p) is the rectangle [aY,aY+kH) x [aX,aX+kW). Near an edge the rectangle shifts inward rather than filling with invalid padding. If both axes fit inside M, the implementation uses a single full-image window. Otherwise, 16 × 16 query tiles are processed in chunks of up to four. A shared key union is masked separately for every query to recover S(p) exactly. This grouping is an execution detail, not four heads or a change to the neighbourhood size.

3.1 What NA changes

The attention weights depend on query/key content and relative position; NA is not a fixed smoothing filter. Overlap occurs between donor neighbourhoods, not between separately predicted outputs. The per-query support removes a fixed shared-window partition from the context operator, but does not guarantee a seam-free model: local windows, cross-scale blocks, convolution borders, and PU2 retain their own boundary behavior. NA complements these components rather than replacing them.

NA specifies which pixels can interact. SDPA and FlexAttention specify how that attention is executed. The optional Flex path preserves the same per-query support and learned Fourier factors; enabling it is not an architectural switch from window attention to NA. Backend changes can nevertheless alter floating-point rounding, latency, and memory use.

4 Fourier bias and regional channel mixing

4.1 Stationary relative Fourier bias

The default context bias has rank 16: eight learned two-dimensional frequencies with per-head cosine and sine amplitudes. For phase $\varphi_f(p)=\omega_f^T p$, define the following two-coordinate factors for each frequency. Their dot product depends only on displacement, including a signed directional component.

$$u_f(p) = [a_f \cos \varphi + b_f \sin \varphi, a_f \sin \varphi - b_f \cos \varphi]$$

$$v_f(p) = [\cos \varphi, \sin \varphi]$$

$$B(p,j) = \Sigma_f [a_f \cos(\omega_f^T(p-j)) + b_f \sin(\omega_f^T(p-j))]. \quad (3)$$

Concatenating $[q/\sqrt{32}, u(p)]$ and $[k, v(j)]$ turns content and bias into one dot product with attention scale set to one. This follows the factor-concatenation interface of RIB [1], but the default bias is an analytic Fourier parameterization, not a coordinate MLP. Frequency shapes do not depend on the window. Frequencies begin at log-spaced magnitudes from $2\pi/128$ to $\pi/2$ with golden-angle directions; cosine amplitudes start at 1/8 and sine amplitudes at zero.

4.2 Regional channel attention

After each context spatial update, a separate channel mixer processes four 32 × 32 grids with offsets (0,0), (0,16), (16,0), and (16,16). Pointwise QKV and depthwise 3 × 3 projections produce per-head arrays of shape 32 x n. Channel correlations resemble transposed attention [4], but are computed regionally. With $\varepsilon=10^{-6}$ and n real pixels, the normalized Gram is:

$$G_{ab} = \Sigma_t Q_{at} K_{bt} / [\sqrt{(\Sigma_t Q_{at}^2 + n\varepsilon)} \sqrt{(\Sigma_t K_{bt}^2 + n\varepsilon)}]$$

$$O = \text{softmax}(\exp(\min(\theta, \ln 100))\, G)\, V. \quad (4)$$

Gram products and softmax use FP32 for low-precision inputs, or FP64 for doubles. Probabilities are cast to the value dtype before multiplication. Separable hat weights blend the four outputs: $w_s(t)=1-|((t+s) \bmod 32)-15.5|/16$. For integer pixels, $w_0+w_{16}=1$, so the four 2D products sum to one. This prevents gain changes from blending weights alone; learned regional outputs can still differ.

Finally, the expansion-two feed-forward residual applies $H=\text{GELU}(W_1 z)$, then $W_2[H+\text{GELU}(\text{DW3x3}(H))]$. Main-layer token normalization uses $\varepsilon=10^{-5}$, with FP32 low-precision statistics. Regional channel mixing is separate from spatial Neighbourhood Attention.

5 Cross-scale self-exemplar retrieval

DUAL retrieves features within the input image across scales, rather than from another frame or external exemplar. Cross-scale self-similarity is a restoration precedent [5]. A coarse RGB image is formed by a fixed separable stride-two filter with taps [-3,-9,29,111,111,29,-9,-3]/256, replicated boundaries, and even-size handling. It reuses the same stem and first two local layers as the fine path.

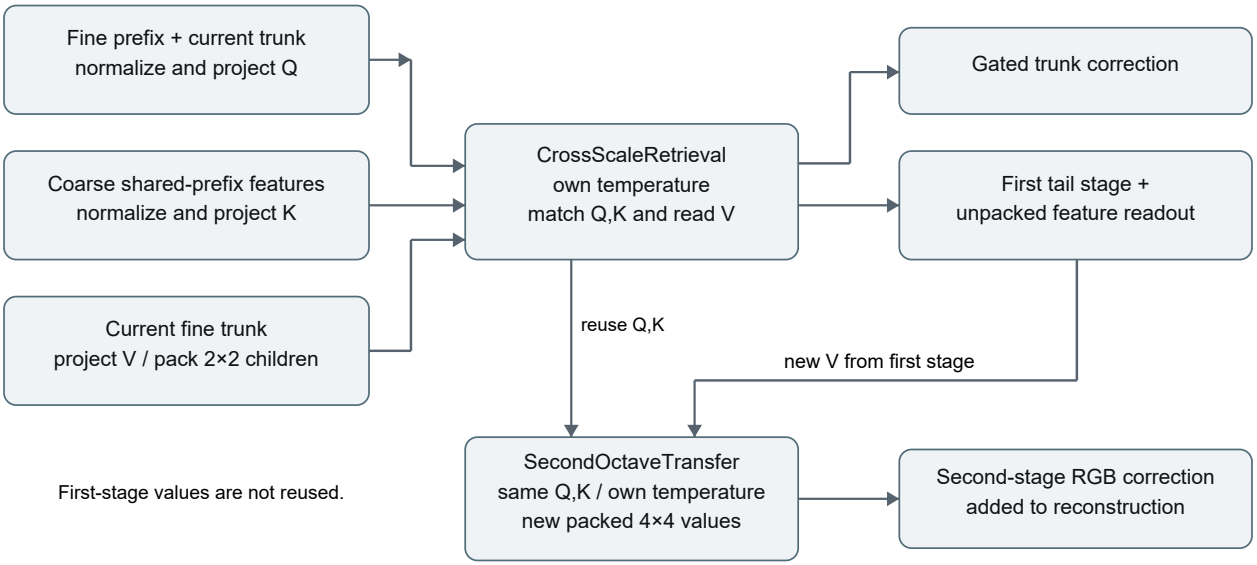


Figure 4. Origins and destinations of cross-scale features. CrossScaleRetrieval provides trunk and first-stage readouts. SecondOctaveTransfer reuses the query/key representations with its own temperature and new values from the first tail stage. The final image skip is omitted for clarity.

The fine path records a prefix after these two layers and continues through the second group. A query projection receives normalized prefix and current features; keys are projected from normalized coarse features. Queries are grouped into blocks of up to 96 × 96 trunk positions with corresponding half-resolution key blocks. Equal-shaped blocks are batched. Retrieval is blockwise, not unrestricted image-wide attention.

$$P_h = \mathrm{softmax}[\tau_h\, \lambda(N_k)\, Q_h K_h{}^T],$$

$$\lambda(N_k) = \ln N_k \,/\, (\ln 1024 \cdot \sqrt{32}),\ 0.1 \le \tau_h \le 10. \tag{5}$$

5.1 CrossScaleRetrieval and SecondOctaveTransfer

CrossScaleRetrieval projects current fine features to C/4 channels. Packing four 2 × 2 child positions produces a 32-channel value vector per head and coarse key. The retrieved tensor supplies a gated trunk correction and, when internal upsampling exists, an unpacked readout into the first 2x reconstruction octave. Both output projections start at zero. This initially leaves the corresponding residual outputs inactive while allowing their readout parameters to learn.

For internal scales of at least four, SecondOctaveTransfer uses the same packed query/key representations but projects new values from that first octave, after its CrossScaleRetrieval readout. Sixteen child positions are packed per key, with two channels per child and head. SecondOctaveTransfer has its own temperature and zero-initialized RGB readout. It therefore does not simply reuse a previously computed attention probability matrix: its values are causally downstream, and its temperature can differ.

Input path	Requested	Internal	First-stage readout	Second-stage RGB
Native	1x	1x	No	No
Native	2x	2x	Yes	No
Native	4x	4x	Yes	Yes
Light PU2	1x	2x	Yes	No
Light PU2	2x	4x	Yes	Yes
Light PU2	4x	8x	Yes	Yes

Table 2. Reconstruction regimes. CrossScaleRetrieval's trunk correction exists independently of its HR readout. Internal 8x uses an additional shuffle for the SecondOctaveTransfer output.

The key-count scaling in (5) changes logit magnitude, not donor identity. It does not establish equivalence between training crops and larger inference images.

6 Reparameterization and dynamic kernels

6.1 Edge-augmented reparameterized convolution

DUAL uses edge-augmented reparameterized convolution (EDBB) at dense stride-one 3 × 3 sites: two independent gain-two 1x1-3x3-1x1 branches, an expanded pointwise path, Sobel-x/Sobel-y/Laplacian edge branches, and an identity when channels match. Depthwise and ordinary pointwise operators are not replaced. Structural reparameterization and edge-oriented branches have established precedents [7,8].

DUAL composes every branch differentiably in kernel space before applying the feature convolution. It does not materialize the gain-two intermediate feature maps. Composition uses FP32 for ordinary parameters and FP64 for double parameters, then casts back for convolution. The construction embeds the original dense operator at initialization in real arithmetic; additional factors can still change optimization. Parameter-space composition is not free: it adds arithmetic, storage, and autograd work.

6.2 Per-image group write-back

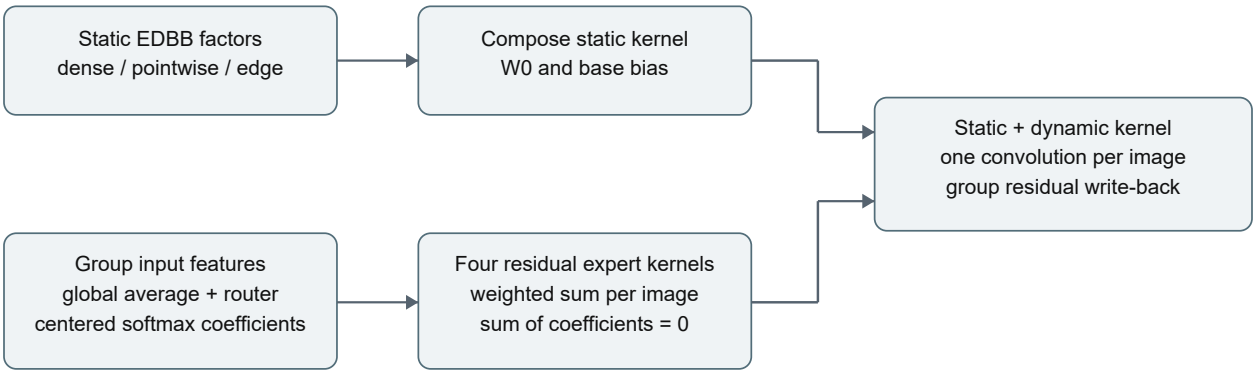


Figure 5. Group-exit dynamic convolution. Static factors compose one base kernel; a global image summary supplies centered coefficients for four residual experts. The effective kernel is mixed before feature convolution. This is not selection among four nonlinear feature-processing networks.

Each group exit adds four zero-initialized residual kernels to the static base. A router forms FP32/FP64 global averages, applies non-affine LayerNorm, and uses a C-to-C/4-to-4 MLP with GELU. The softmax is centered around the uniform mixture, yielding signed coefficients shared across the image.

$$c(x) = \mathrm{softmax}(\mathrm{MLP}(\mathrm{LN}(\mathrm{GAP}(x)))) - 1/K, K=4$$

$$W(x) = W_0 + \Sigma_k\, c_k(x) E_k,\ y = \mathrm{Conv}(x; W(x), b_0). \quad (6)$$

Convolution is linear in its kernel, so premixing is algebraically equivalent to summing the base response and coefficient-weighted expert responses. For B>1, the implementation packs samples into a convolution with groups=B and then restores the batch. These groups isolate images; the channel convolution remains dense. This is continuous kernel mixing, not sparse top-k routing among nonlinear networks [6].

Initially E_k=0, so the dynamic correction vanishes. Expert gradients can differ through c_k, while the router's data gradient through this zero bank initially vanishes. Because Σc_k=0, adding one common kernel to every expert changes nothing. This is a contrast-space redundancy, not a rank bound on the effective convolution.

6.3 What deployment can fold

prepare_for_export() folds static factors in FP64 on an eval copy and retains input-dependent routing and experts. Freezing a mixed kernel observed on one image would change the model. Folding removes the training parameterization and is not a reversible training operation. Algebraic equivalence does not imply bitwise equality after low-precision casts or backend changes.

7 Training execution and deployment

Option	Default	Role
context_na	True	Neighbourhood Attention in context layers
use_checkpoint	True	Selective whole-layer activation checkpointing
attn_type	masked	SDPA for NA; hybrid opts into eligible compiled Flex calls
g_attention_impl	sdpa	Eager local attention path
compile_g_attention_impl	direct	Eligible local path after preparation

Table 3. Effective execution defaults for all presets and scales. Activation checkpointing is distinct from saving training state. NA remains enabled under both attention backend choices.

prepare_for_compile(input_shape, input_dtype) configures an undeployed training model using capability checks shared by every preset and output scale. With FP16/BF16 inputs, eligible full G16/G32 anchored dot-product layers can use the bundled dense-relative-bias Triton kernel. Device, dtype, geometry, and mask checks still apply at execution. Unsupported local geometry routes to SDPA with a warning; missing direct-kernel dependencies are reported rather than silently ignored. Setting compile_g_attention_impl to sdpa keeps local attention on SDPA.

For eligible CUDA BF16 context training with deterministic algorithms disabled, images are folded into the head dimension so one tile mask is shared across images. With attn_type=masked, this path uses SDPA and prefers cuDNN attention, then efficient attention or the math implementation according to backend support. The 32 content channels and 16 Fourier lanes give Q/K width 48, while V stays at width 32. The standard layout instead zero-pads V and slices the result. These layouts preserve the same neighbourhoods and relative coordinates.

7.1 Optional FlexAttention backend

attn_type=hybrid replaces only eligible masked NA calls during compiled CUDA BF16 training. The Flex path requires context_na=True, the PyTorch FlexAttention/Triton backend, rank-16 Fourier bias, a 32- or 64-pixel context, and deterministic algorithms disabled. The same four-tile geometry is encoded in a BlockMask; Q/K are zero-padded from width 48 to 64 and V remains width 32. Learned Fourier factors remain differentiable Q/K operands. Local-window and cross-scale attention are not replaced; unmasked full-image context and eager/evaluation NA continue to use SDPA. Other dtypes and deterministic mode retain SDPA for NA.

Before compiling, move the training model to its target CUDA device and call prepare_for_compile with the actual positive BCHW RGB input shape and input_dtype=torch.bfloat16. The hook prepares reusable geometry masks, accounting for Light's PU2 padding. Re-run preparation after a device/dtype move or input-shape change; an eligible hybrid call with a missing mask raises an error. The flag alone does not compile the model. Flex and SDPA preserve the attention definition, not guaranteed bitwise equality or a fixed speedup.

7.2 Selective whole-layer checkpointing

use_checkpoint=True enables one non-reentrant checkpoint boundary per residual layer, including executions of the shared fine/coarse prefix. Within each boundary, the policy retains recognized fused SDPA, FlexAttention, and bundled direct-attention operation results, while preferring recomputation for other operations. QKV projection, gathering, normalization, gating, channel mixing, and feed-forward work are inside the layer boundary rather than a separate nested context checkpoint. Actual memory and recomputation depend on the selected backend and compiler graph.

use_checkpoint is the only activation-checkpoint option: False disables explicit checkpointing. Checkpoints are applied only in training with gradients enabled, not during evaluation or export. The eager AMP recomputation scope disables autocast's weight cache so shared-prefix weights are recast consistently; the recomputation safeguard does not reduce the requested arithmetic precision.

7.3 Evaluation and export

Evaluation NA uses SDPA, not the compiled-training Flex path, and disables activation checkpointing. Eager inference without gradients reuses bounded integer-coordinate caches and computes shared key-side Fourier factors once per layer forward; learned factors are not cached across weight updates. SDPA is a dispatch API, not a guarantee that FlashAttention is available or selected. Mask, dtype, shape, platform, and backend support govern the implementation.

The architecture file bundles its optional direct local-attention kernels and can import without traiNNer. Export preparation resets spatial routing to SDPA, disables the BF16 training layout, clears Flex masks, and folds static convolution factors. NA support remains unchanged. This prepares a model copy; it does not itself generate ONNX, build a TensorRT engine, or package a TensorRT plugin. TensorRT fusion and any external NA plugin are separate deployment choices, not enabled by attn_type=hybrid.

Call model.eval(), make a separate deep copy, then call prepare_for_export() on that copy. The DUAL method returns None. Keep the unfolded model and checkpoint for training or resumption.

8 Integration and limitations

8.1 Checkpoints and integration

Select the factory, requested scale, and structural options before loading an unfolded state dictionary. Load strictly when the architecture matches. Settings such as context_na and window sizes are not fully described by parameter shapes, so store the architecture configuration alongside the weights.

For traiNNer integration, place the single file in traiNNer/archs and select its registered factory in network_g. Avoid simultaneously importing an older module that registers the same names. Keep precision, color convention, output clipping, padding, and tiling choices explicit in inference tooling. The architecture itself does not clamp its RGB output to a display range.

8.2 Finite-image and numerical limits

Neighbourhood Attention is bounded; it is not unrestricted global attention. PU2 changes phase handling and the relation between trunk pixels and input pixels. Local-window ownership, cross-scale block boundaries, convolution padding, and image-wide routing prevent a blanket claim of exact translation equivariance. Independently tiled inference changes both available donors and routing statistics, so it is not generally identical to whole-image inference. Parameter-compatible changes to support can also change texture behavior.

Kernel composition, factor concatenation, and layout changes preserve the stated algebra, but floating-point accumulation and casts can affect outputs and gradients. This report describes the architecture; it does not present quality or runtime benchmarks.

8.3 Terminology

Notation	Meaning
NA	Neighbourhood Attention: a content-weighted donor neighbourhood selected per query pixel.
G16 / C32 / C64	Local window 16 / NA context support 32 or 64.
SDPA / Flex	Execution backends; neither changes NA's donor-support definition.
RFB / RPB	Relative Fourier bias / tabulated relative position bias.
PU2 / F64	PixelUnshuffle by 2 / reconstruction width of 64 channels.
Q / K / V	Query / key / value projections; their origins are specified for each operator.
Octave	One 2x spatial-resolution step.

References

[1] D. Lee, S. Yun, J. Im, and Y. Ro. Rank-Factorized Implicit Neural Bias: Scaling Super-Resolution Transformer with FlashAttention. 2026. arXiv:2603.06738.

[2] J. Liang et al. SwinIR: Image Restoration Using Swin Transformer. ICCV Workshops, 2021. arXiv:2108.10257.

[3] A. Hassani et al. Neighborhood Attention Transformer. CVPR, 2023. arXiv:2204.07143.

[4] S. W. Zamir et al. Restormer: Efficient Transformer for High-Resolution Image Restoration. CVPR, 2022. arXiv:2111.09881.

[5] Y. Mei et al. Image Super-Resolution with Cross-Scale Non-Local Attention and Exhaustive Self-Exemplars Mining. CVPR, 2020. arXiv:2006.01424.

[6] Y. Chen et al. Dynamic Convolution: Attention over Convolution Kernels. CVPR, 2020. arXiv:1912.03458.

[7] X. Ding, X. Zhang, J. Han, and G. Ding. Diverse Branch Block: Building a Convolution as an Inception-like Unit. CVPR, 2021. arXiv:2103.13425.

[8] X. Zhang, H. Zeng, and L. Zhang. Edge-oriented Convolution Block for Real-time Super Resolution on Mobile Devices. ACM Multimedia, 2021. Official ECBSR repository.